ODETTE Python interface

Bindings_Python-interface folder

Project Overview

This project provides a Python interface for satellite orbital mechanics, leveraging C++ bindings for high-performance calculations.

Core Python Modules

File	Purpose
setup.py	Project installation configuration
orbit_toolkit.py	Orbital calculations and operations toolkit
satellite_core_structure.py	Defines core satellite structure and functionality
gauss_tle_test.py	Two-Line Element (TLE) calculation tests using Gauss method

Configuration Files

File	Function
pyproject.toml	Modern Python project configuration
CMakeLists.txt	CMake build system configuration

Logging and Results

File	Description
log_output.txt	Project output logs
gauss_tle_test_results/	Test result storage
log_orbital_elem/	Orbital element logging

Technical Deep Dive: Python–C++ Bindings

The project implements a sophisticated interface between Python and C++:

satellite_core_bindings.cpp

- Creates Python bindings for C++ satellite core functionality
- Enables high-performance computations
- Provides seamless Python-C++ integration for PyBind11

Orbital Mechanics Capabilities

Key functionalities include:

- TLE (Two-Line Element) data parsing
- Orbital element calculations
- Satellite trajectory analysis

Getting Started

1. Ensure all dependencies are installed
2. Review setup.py for installation requirements
3. Use pyproject.toml for project configuration

Development Notes

- Performance-critical calculations implemented in C++
- Pythonic interface for ease of use
- Modular design for extensibility

ORBIT_TOOLKIT.PY

Python file that provides Python interface classes and JavaGUI Bridge class. In the following we shed some light, in a more detailed manner, on the provided classes.

OrbitVisualizer Class

Purpose

The `OrbitVisualizer` class is designed to create 3D visualizations of satellite orbits. It provides methods to plot and visualize different types of orbital trajectories.

Key Methods and Functionality

1. `__init__()`:
   - Initializes a figure and axis for plotting
   - Sets Earth's radius to 6378.0 kilometers
2. `_setup_plot()`:
   - Creates a 3D matplotlib figure
   - Plots a simplified representation of the Earth (blue, semi-transparent sphere)
   - Sets axis labels and title
   - Ensures equal aspect ratio for the 3D plot
3. `_plot_orbit()`:
   - Plots a trajectory in 3D space
   - Allows customization of color, line width, and label
4. `visualize_tle()`:
   - Visualizes an orbit using Two-Line Element (TLE) data
   - Propagates the orbit over a specified time span
   - Converts positions to Earth-Centered Inertial (ECI) coordinates
   - Normalizes positions to Earth radii
   - Optionally saves and shows the plot
5. `visualize_rk45()`:
   - Visualizes an orbit using RK45 numerical integration
   - Parses Tracking Data Message (TDM) files
   - Supports different perturbation models (J2, third-body, solar radiation, atmospheric drag)
   - Propagates orbit and converts to ECI coordinates
   - Similar saving and plotting options as `visualize_tle()`
6. `compare_orbits()`:

- Compares TLE and RK45 orbital propagations
- Plots multiple trajectories on the same 3D graph
- Supports different perturbation settings
- Optional visualization and saving of the plot

OrbitPropagator Class

OVERVIEW

The `OrbitPropagator` class (inheriting from `OrbitVisualizer`) focuses on computing and analyzing orbital trajectories using different propagation methods.

Key Methods and Functionality

1. `propagate_tle()`:
   - Propagates an orbit using TLE data
   - Computes positions and velocities at different time steps
   - Calculates orbital elements:
     - Semi-major axis
     - Eccentricity
     - Inclination
     - Right Ascension of Ascending Node (RAAN)
     - Argument of Perigee
     - True Anomaly
     - Mean Anomaly
2. `propagate_rk45()`:
   - Propagates an orbit using RK45 numerical integration
   - Parses TDM files to get initial conditions
   - Supports various perturbation models
   - Similar orbital element calculations as `propagate_tle()`
3. `compare_orbital_elements()`:
   - Comprehensive method for comparing orbital trajectories
   - Can compare:
     - TLE vs RK45 propagation
     - Multiple RK45 propagations with different perturbation settings
   - Computes and visualizes:
     - Positions and velocities
     - Orbital elements
     - Differences between trajectories
   - Generates statistical comparisons and plots

4. `set_perturbation_parameters()`:
    - Allows dynamic setting of perturbation parameters
    - Supports parameters for:
        - Solar Radiation Pressure (SRP)
        - Atmospheric Drag
        - Other orbital perturbation factors
5. `get_perturbation_parameters()`:
    - Retrieves current perturbation parameter values

JavaGUIBridge Class

Overview

This code defines a `JavaGUIBridge` class that serves as an interface between Java and Python code for an "Orbit Toolkit" application. It allows Java code to communicate with Python-based orbit visualization and propagation tools.

Key components

Class structure

```
class JavaBridge:
    """Enhanced bridge class for Java-Python interface for the Orbit
Toolkit."""

    def __init__(self):
        self.visualizer = OrbitVisualizer()
        self.propagator = OrbitPropagator()
        self.status_callback = None
```

The class initializes two main components:

- `OrbitVisualizer`: For visualizing orbital trajectories
- `OrbitPropagator`: For calculating (propagating) orbital paths

Communication Mechanism

The bridge uses JSON for communication between Java and Python:

1. The Java side sends commands as JSON strings
2. Python parses these commands, executes them, and returns results as JSON

Command Handling

The central method is `execute_command()`, which:

1. Parses the incoming JSON command
2. Determines which function to call based on the "command" field
3. Calls the appropriate handler method
4. Returns the results as a JSON string

Command Types

The bridge supports several command types:

- `visualize_tle`: Visualize orbits using Two-Line Element (TLE) data
- `visualize_rk45`: Visualize orbits using RK45 integration
- `compare_orbits`: Compare orbits calculated by different methods
- `propagate_tle`: Compute orbital propagation using TLE
- `propagate_rk45`: Compute orbital propagation using RK45 integration
- `compare_orbital_elements`: Compare orbital elements between methods
- `set_perturbation_params`: Set parameters for orbital perturbations
- `get_perturbation_params`: Get current perturbation parameters

Status Updates

The bridge provides progress updates through a callback mechanism:

```
def set_status_callback(self, callback_func):
    """Set a callback function to receive status updates"""
    self.status_callback = callback_func
```

The Java side can register a callback function that will receive progress updates as operations execute.

Error Handling

The code includes comprehensive error handling:

```
def _create_error_response(self, message, error_code=None):
    """Create a standardized error response"""
    response = {
        'status': 'error',
        'message': message
    }
    if error_code:
        response['error_code'] = error_code
    return json.dumps(response)
```

Each handler method has try-except blocks to catch and report errors.

Data Conversion

A special method converts Python data structures to JSON-serializable formats:

```
def _convert_to_serializable(self, obj):
    """Convert numpy arrays, lists, and other objects to JSON-serializable
formats"""
    # …
```

This is particularly important for NumPy arrays and other scientific data structures.

Typical Workflow

1. The Java GUI creates a command as a JSON object
2. Java sends the command to Python via this bridge
3. The bridge parses the command and calls the appropriate handler
4. The handler executes orbit calculations or visualizations
5. Results are converted to JSON and returned to Java
6. Progress updates are sent via the callback during execution

Domain-Specific Features

The code deals with orbital mechanics and includes functionality for:

- TLE (Two-Line Element) orbital data format
- RK45 (Runge-Kutta) numerical integration
- TDM (Tracking Data Message) format
- Perturbation modeling for orbit calculations
- Calculation of various orbital elements and positions